評価値(評価コスト$\hat{f}(v) = \hat{g}(v) + \hat{h}(v)$)を用いた探索

準備

- $\hat{g}(v)$ は s～v のコスト(の暫定値), $\hat{h}(v)$ は v～g のコスト(の推定値)等.
- […] はノード(状態)のリストを表す.
- 変数 OL, CL, V_X はノードのリスト, 変数 X は単一のノードを保持する.
- s は初期状態を表す.
- +はリスト連結の演算子とする.
- pop(l)は, リスト l の先頭(左端)から要素を一つとりだす関数とする.
- next(v) は, v の隣接ノード(v から遷移可能な状態)のリストを返す関数とする. その際, 各ノードに v へのリンクを持たせる. (探索木や経路を生成するのに必要)
- sort(l)は, リスト l の要素を, その評価コスト($\hat{f}(v) = \hat{g}(v) + \hat{h}(v)$)で昇順に並べ替えたリストを返す関数とする.
- uniq(l)は, リスト l の重複を(最左の要素のみを残すことによって)取り除いたリストを返す関数とする.

分岐限定探索 ($\hat{f}(v) = \hat{g}(v)$, $(\hat{h}(v) \equiv 0)$)

```
1   OL ← [s]; CL ← [];
2   if (OL が空) 終了(失敗);
3   X ← pop(OL);
4   if (X が目標状態) 終了(成功);
5   CL ← CL + [X];
6   V_X ← next(X);
7   V_X の要素の各々(v)について, 以下を行う.
      7.1   if (v が CL に含まれる) V_X から v を取り除く; // 探索済みの状態を除外
8   OL ← sort(OL + V_X);        // 評価値(評価コスト)の昇順に並べ替える
9   OL ← uniq(OL);              // 重複は, 評価値(評価コスト)の悪いものを除去
10  goto 2
```

※ 7.1 で v が CL に含まれている場合, V_X 中の v の$\hat{g}(v)$は常に CL 中の v の$\hat{g}(v)$以上になる.

※ 8 の並べ替えを除けば, グラフの(評価コストを用いない)探索に類似している.

ヒューリスティック探索 ($\hat{f}(v) = \hat{h}(v)$, $(\hat{g}(v) \equiv 0)$)

```
1   OL ← [s]; CL ← [];
2   if (OL が空) 終了(失敗);
3   X ← pop(OL);
4   if (X が目標状態) 終了(成功);
5   CL ← CL + [X];
6   V_X ← next(X);
7   V_X の要素の各々(v)について, 以下を行う.
      7.1   if (v が CL または OL に含まれる) V_X から v を取り除く;
                                  // 探索済みの状態, および重複を除外
8   OL ← sort(OL + V_X);        // 評価値(評価コスト)の昇順に並べ替える
9   goto 2
```

※　評価値 $\hat{h}(v)$ は，常に変化しないので，重複の除去はどれを取り除いてもよい．

※　8 の並べ替えを除けば，グラフの(評価コストを用いない)横型探索に類似している．

一般の最小コストパス探索（$\hat{f}(v) = \hat{g}(v) + \hat{h}(v)$）

```
1   OL ← [s]; CL ← [];
2   if (OL が空) 終了(失敗);
3   X ← pop(OL);
4   if (X が目標状態) 終了(成功);
5   CL ← CL + [X];
6   V_X ← next(X);
7   V_X の要素の各々(v)について，以下を行う．
    7.1   if (v が CL に含まれ，かつ，V_X 中の v の f̂(v)が CL 中の v の f̂(v)以上である)
              V から v を取り除く;              // 探索済みの状態を除外
    7.2   if (v が CL に含まれ，かつ，V_X 中の v の f̂(v)が CL 中の v の f̂(v)より小さい)
              CL から v を取り除く;   // s～v のコスト ĝ(v)が更新された場合，
                                                // v を CL から OL に戻す
8   OL ← sort(OL + V_X);        // 評価値(評価コスト)の昇順に並べ替える
9   OL ← uniq(OL);              // 重複は，評価値(評価コスト)の悪いものを除去
10  goto 2
```

※ V_X 中の v の $\hat{f}(v)$が CL 中の v の $\hat{f}(v)$より小さい場合，$\hat{h}(v)$は変化しないので，$\hat{g}(v)$が更新されたことになる．

※ $\hat{g}(v)$が更新された場合，先に v を経由して探索された状態を再び探索し直す必要が生じるため，7.2 のように，v を再び OL に戻す必要がある．

最急降下法(山登り法)（$\hat{f}(v) = \hat{h}(v)$, $(\hat{g}(v) \equiv 0)$）

※　変数 V は，ノード(状態)のリストではなく，1 つのノード(状態)を表す．
　　また，リンクはすべて双方向であるとする．

```
1   X ← s;
2   V ← pop(sort(next(X)));   // X の隣接ノードのうちで，評価値(評価コスト)の
                              // 最も小さいものを V に代入する．
3   if (f̂(V)≧f̂(X)) 終了(X が極小);
4   X ← V
5   goto 2
```